

Πλήρης Ανάλυση Κώδικα για Αυτο-οδηγούμενο Αυτοκίνητο

Αγαπητοί φοιτητές,

Ας επανεξετάσουμε τον κώδικα με τη δέουσα προσοχή, εστιάζοντας σε κάθε αρχείο ξεχωριστά και εμβαθύνοντας στις λειτουργίες, τους σκοπούς και τα μαθηματικά που κρύβονται πίσω από κάθε γραμμή. Αυτή η ανάλυση είναι πλήρης και καλύπτει όλα τα αρχεία του φακέλου.

1. README

- **Σκοπός:** Αυτό το αρχείο δεν είναι κώδικας, αλλά ένα εγχειρίδιο οδηγιών. Είναι απαραίτητο για κάθε έργο, καθώς καθοδηγεί τον προγραμματιστή και τον χρήστη.
- **Λειτουργία:** Μας πληροφορεί για το πώς να χρησιμοποιήσουμε τις συναρτήσεις `save()` και `discard()` στην κονσόλα του browser. Αυτό υποδεικνύει ότι το πρόγραμμα χρησιμοποιεί τον `localStorage` του browser για αποθήκευση, ένα κρίσιμο στοιχείο για τη διατήρηση του εκπαιδευμένου νευρωνικού δικτύου. Μας προτρέπει επίσης να πειραματιστούμε με τις μεταβλητές `N` (αριθμός αυτοκινήτων) και το `mutation amount` (ποσοστό μετάλλαξης), οι οποίες είναι οι κύριες παράμετροι του γενετικού αλγορίθμου. Τέλος, μας προειδοποιεί να χρησιμοποιούμε το `discard()` αν αλλάζουμε την αρχιτεκτονική του δικτύου ή του αισθητήρα, για να αποφύγουμε ασυμβατότητες.

2. index.html

- **Σκοπός:** Η δομική βάση της εφαρμογής, ο σκελετός.
- **Ανάλυση:**
 - `<meta charset="utf-8" />`: Διασφαλίζει τη σωστή εμφάνιση όλων των χαρακτήρων.
 - `<title>Self Driving Car</title>`: Ο τίτλος της καρτέλας του browser.
 - `<link rel="stylesheet" href="style.css" />`: Σύνδεση με το αρχείο στυλ.
 - `<canvas id="carCanvas"></canvas><canvas id="networkCanvas"></canvas>`: Ορίζονται δύο καμβάδες. Ο πρώτος (`carCanvas`) για την προσομοίωση του δρόμου και των αυτοκινήτων, και ο δεύτερος (`networkCanvas`) για την οπτικοποίηση του νευρωνικού δικτύου.
 - **Σειρά Φόρτωσης Scripts:** Αυτή είναι η πιο κρίσιμη πτυχή του αρχείου. Τα αρχεία JavaScript φορτώνονται με μια συγκεκριμένη σειρά εξάρτησης (`dependency`).
 1. `visualizer.js`, `network.js`, `sensor.js`, `utils.js`, `road.js`, `controls.js`, `car.js`: Αυτά τα αρχεία ορίζουν κλάσεις και συναρτήσεις.
 2. `main.js`: Αυτό το αρχείο χρησιμοποιεί όλες τις παραπάνω κλάσεις και πρέπει να φορτωθεί τελευταίο. Αν αλλάξετε τη σειρά, θα προκύψουν σφάλματα αναφοράς (`Reference Errors`).

3. style.css

- **Σκοπός:** Καθορίζει την εμφάνιση της σελίδας και των καμβάδων.
- **Ανάλυση:**
 - `body { ... }`: Εφαρμόζει κανόνες σε ολόκληρο το σώμα της σελίδας, όπως μηδενικά

περιθώρια (margin: 0;), σκούρο γκρι φόντο (background: darkgray;) και απόκρυψη της κύλισης (overflow: hidden;).

- `#carCanvas { background: lightgray; }`: Καθορίζει το φόντο του καμβά των αυτοκινήτων σε ανοιχτό γκρι.
- `#networkCanvas { background: black; }`: Καθορίζει το φόντο του καμβά του νευρωνικού δικτύου σε μαύρο.
- **Αιτιολογία:** Η χρήση του `overflow: hidden;` είναι σημαντική για να μην εμφανίζονται μπάρες κύλισης όταν το περιεχόμενο ξεπερνά τις διαστάσεις του παραθύρου, κάτι που είναι συνηθισμένο σε εφαρμογές πλήρους οθόνης.

4. utils.js

Αυτό το αρχείο περιέχει τις θεμελιώδεις μαθηματικές συναρτήσεις.

function lerp(a, b, t)

- **Λογική:** `return a + (b - a) * t;`
- **Μαθηματικά:** Η φόρμουλα για τη γραμμική παρεμβολή είναι $P(t)=a+t(b-a)$.
 - Η t είναι ένας συντελεστής που κυμαίνεται από 0 (αρχικό σημείο) έως 1 (τελικό σημείο). Αυτή η συνάρτηση μας επιτρέπει να κινούμαστε ομαλά μεταξύ δύο τιμών, κάτι που είναι απαραίτητο για animations, μεταβάσεις και γενετικούς αλγορίθμους.

function getIntersection(A, B, C, D)

- **Σκοπός:** Υπολογίζει το σημείο τομής δύο ευθειών.
- **Μαθηματικά:** Η συνάρτηση λύνει ένα σύστημα δύο γραμμικών εξισώσεων με δύο αγνώστους, t και u , που αντιπροσωπεύουν την παραμετρική μορφή των δύο ευθειών.
 - $(A+t(B-A))=(C+u(D-C))$
 - Ο κώδικας υπολογίζει τους αριθμητές (t_{Top} , u_{Top}) και τον κοινό παρονομαστή (bottom) της λύσης. Αν ο παρονομαστής είναι μηδέν, οι ευθείες είναι παράλληλες και δεν τέμνονται.
 - Το `if (t >= 0 && t <= 1 && u >= 0 && u <= 1)` ελέγχει αν το σημείο τομής βρίσκεται **εντός των ευθύγραμμων τμημάτων**, όχι απλώς στις ευθείες που τα περιέχουν.
- **Χρήση:** Είναι το βασικό δομικό στοιχείο για την ανίχνευση συγκρούσεων.

function polysIntersect(poly1, poly2)

- **Σκοπός:** Ελέγχει αν δύο πολύγωνα τέμνονται.
- **Λογική:** Ο αλγόριθμος είναι επαναληπτικός. Ελέγχει κάθε ευθύγραμμο τμήμα (edge) του πρώτου πολυγώνου με κάθε ευθύγραμμο τμήμα του δεύτερου πολυγώνου, χρησιμοποιώντας τη συνάρτηση `getIntersection`. Αν βρει έστω και μία τομή, επιστρέφει `true`.
- **Γιατί είναι απαραίτητο;** Το αυτοκίνητο και τα εμπόδια είναι πολύγωνα, όχι απλά σημεία. Η χρήση αυτής της συνάρτησης επιτρέπει ακριβείς υπολογισμούς σύγκρουσης.

function getRGBA(value)

- **Σκοπός:** Μετατρέπει μια αριθμητική τιμή σε ένα χρώμα RGBA.

- **Λογική:** Η τιμή της παραμέτρου (value) χρησιμοποιείται για να καθορίσει το χρώμα και τη διαφάνεια (alpha). Αν η τιμή είναι αρνητική, το χρώμα είναι μπλε ($B = 255$), αν είναι θετική, είναι κόκκινο ($R = 255$). Το απόλυτο μέγεθος της τιμής καθορίζει τη διαφάνεια.
- **Χρήση:** Αυτή η συνάρτηση είναι κλειδί για την οπτικοποίηση των βαρών και των προκαταλήψεων του νευρωνικού δικτύου, καθώς μας δίνει μια άμεση οπτική αναπαράσταση της λειτουργίας τους.

5. controls.js

- **Σκοπός:** Διαχειρίζεται τις εντολές του οδηγού, είτε από τον άνθρωπο είτε από την A.I.
- **Ανάλυση:**
 - `constructor(type)`: Δέχεται μια παράμετρο `type`.
 - `case "KEYS": this.#addKeyboardListeners()`;: Αν ο τύπος είναι "KEYS", ενεργοποιεί τους ακροατές για τα πλήκτρα του πληκτρολογίου (`ArrowUp`, `ArrowDown`, κ.λπ.).
 - `case "DUMMY": this.forward = true`;: Αν ο τύπος είναι "DUMMY", το αυτοκίνητο κινείται συνεχώς προς τα εμπρός. Αυτός ο τύπος χρησιμοποιείται για τα αυτοκίνητα της κυκλοφορίας, τα οποία δεν χρειάζονται χειρισμό.

6. road.js

- **Σκοπός:** Δημιουργεί και σχεδιάζει τον δρόμο.
- **Ανάλυση:**
 - `constructor(x, width, laneCount = 3)`: Ορίζει την κεντρική θέση, το πλάτος και τον αριθμό των λωρίδων.
 - **Καμπύλες:** Η δημιουργία των ορίων του δρόμου (`this.borders`) γίνεται με μια βρόχο και τη συνάρτηση `Math.sin`. Αυτό δίνει την εντύπωση ενός καμπυλωτού δρόμου, αντί για μια απλή ευθεία.
 - `const x = Math.sin(y * 0.01) * 50`;: Η τιμή `x` μεταβάλλεται αρμονικά με βάση την τιμή του `y`, δημιουργώντας μια ημιτονοειδή καμπύλη.
 - `getLaneCenter(laneIndex)`: Μια πολύ χρήσιμη μέθοδος που υπολογίζει το κέντρο της κάθε λωρίδας.
 - `draw(ctx)`: Σχεδιάζει τον δρόμο στον καμβά. Χρησιμοποιεί το `ctx.setLineDash([20, 20])` για να δημιουργήσει τις διακεκομμένες γραμμές της λωρίδας.

7. car.js

- **Σκοπός:** Η καρδιά της προσομοίωσης, η κλάση που αντιπροσωπεύει το αυτοκίνητο.
- **Ανάλυση:**
 - `constructor(...)`: Εκτός από τις ιδιότητες κίνησης, προστέθηκαν νέες:
 - `this.damaged = false`;: Παρακολουθεί την κατάσταση σύγκρουσης.
 - `this.useBrain = controlType == "AI"`;: Αποφασίζει αν το αυτοκίνητο θα ελέγχεται από την A.I.
 - **update(roadBorders, traffic)**:
 1. `if (!this.damaged)`: Αν το αυτοκίνητο δεν έχει συγκρουστεί, τότε μόνο εκτελείται ο κώδικας κίνησης και ανίχνευσης.

2. `this.#assessDamage(...)`: Ελέγχει αν το αυτοκίνητο έχει συγκρουστεί με βάση τα όρια του δρόμου και τα άλλα αυτοκίνητα.
3. `this.sensor.update(...)`: Ο αισθητήρας ενημερώνεται με τα τελευταία δεδομένα.
4. **A.I. Logic**:
 - `const offsets = this.sensor.readings.map(...)`: Οι ενδείξεις του αισθητήρα (αποστάσεις) μετατρέπονται σε τιμές από 0 έως 1.
 - `const outputs = NeuralNetwork.feedForward(offsets, this.brain);` Οι τιμές αυτές δίνονται ως είσοδο στον "εγκέφαλο" (νευρωνικό δίκτυο).
 - `if (this.useBrain) { ... }`: Αν το αυτοκίνητο είναι A.I., η έξοδος του δικτύου καθορίζει τις εντολές χειρισμού.

8. sensor.js

- **Σκοπός**: Ο "αισθητήρας" του αυτοκινήτου, που ανιχνεύει το περιβάλλον.
- **Ανάλυση**:
 - `this.rayCount, this.rayLength, this.raySpread`: Καθορίζουν την ακρίβεια και την εμβέλεια του αισθητήρα.
 - **#castRays(...)**: Δημιουργεί τις ακτίνες με βάση τη θέση και τη γωνία του αυτοκινήτου.
 - **#getReading(...)**:
 - **Λογική**: Ελέγχει για τομές μεταξύ κάθε ακτίνας και όλων των εμποδίων (ορίων δρόμου, άλλων αυτοκινήτων).
 - **Γιατί η πιο κοντινή**: Χρησιμοποιεί `Math.min(...offsets)` για να βρει το μικρότερο offset, δηλαδή το πιο κοντινό σημείο τομής. Αυτό είναι κρίσιμο, καθώς το αυτοκίνητο πρέπει να αντιδρά στο πιο κοντινό εμπόδιο, όχι σε κάποιο που βρίσκεται πιο μακριά.

9. network.js

- **Σκοπός**: Η κλάση που υλοποιεί το νευρωνικό δίκτυο, τον "εγκέφαλο" της A.I.
- **Ανάλυση**:
 - `class NeuralNetwork`:
 - `constructor(neuronCounts)`: Δημιουργεί μια σειρά από επίπεδα (Level) με βάση τον αριθμό των νευρώνων σε κάθε επίπεδο.
 - **static feedForward(...)**: Είναι μια στατική μέθοδος, δηλαδή μπορεί να κληθεί απευθείας από την κλάση χωρίς να χρειάζεται να δημιουργηθεί αντικείμενο.
 - **Μαθηματικά**: Εφαρμόζει τη διαδικασία **feed-forward**, όπου οι εισόδοι κάθε επιπέδου πολλαπλασιάζονται με τα βάρη, αθροίζονται και στη συνέχεια περνούν από μια συνάρτηση ενεργοποίησης (activation function).
 - **static mutate(...)**: Αυτός είναι ο πυρήνας του **γενετικού αλγορίθμου**.
 - **Λογική**: Εφαρμόζει τυχαίες αλλαγές (mutations) στα βάρη και τις προκαταλήψεις του δικτύου, χρησιμοποιώντας τη συνάρτηση `lerp`. Η τιμή `amount` καθορίζει την ένταση της μετάλλαξης.
 - `class Level`:
 - **static feedForward(...)**: Υπολογίζει τις εξόδους για ένα συγκεκριμένο επίπεδο.

- **Συνάρτηση Ενεργοποίησης:** Ο κώδικας χρησιμοποιεί μια απλή δυαδική συνάρτηση (step function): αν το άθροισμα των εισόδων πολλαπλασιασμένο με τα βάρη είναι μεγαλύτερο από την προκατάληψη (bias), η έξοδος είναι 1 (ενεργοποίηση), αλλιώς 0 (απενεργοποίηση).

10. visualizer.js

- **Σκοπός:** Οπτικοποιεί το νευρωνικό δίκτυο για εκπαιδευτικούς και διαγνωστικούς σκοπούς.
- **Ανάλυση:**
 - **static drawNetwork(...):** Η κύρια μέθοδος που καλεί τη drawLevel για κάθε επίπεδο.
 - **static drawLevel(...):** Σχεδιάζει τους νευρώνες (κύκλοι) και τις συνδέσεις (γραμμές) μεταξύ τους.
 - `ctx.strokeStyle = getRGBA(weights[i][j]);`: Αυτή η γραμμή είναι ενδεικτική της οπτικής νοημοσύνης του κώδικα.
 - **Οπτικοποίηση Βαρών:** Το χρώμα της γραμμής που συνδέει δύο νευρώνες εξαρτάται από την τιμή του βάρους. Αν το βάρος είναι θετικό, η γραμμή είναι κόκκινη. Αν είναι αρνητικό, είναι μπλε. Η διαφάνεια του χρώματος δείχνει πόσο ισχυρό είναι το βάρος. Με αυτό τον τρόπο, μπορούμε να δούμε πώς λειτουργεί ο "εγκέφαλος" του αυτοκινήτου σε πραγματικό χρόνο.

11. main.js

- **Σκοπός:** Ο κεντρικός ελεγκτής της εφαρμογής, ο οποίος συντονίζει όλες τις κλάσεις και εκτελεί τον αλγόριθμο.
- **Ανάλυση:**
 - **Αρχικοποίηση:**
 - `const N = 100;`: Ορίζει τον αριθμό των αυτοκινήτων για την εκπαίδευση.
 - `const traffic = [...];`: Δημιουργεί τα αυτοκίνητα της κυκλοφορίας ως DUMMY τύπους.
 - `if (localStorage.getItem("bestBrain")) { ... }`: Ελέγχει για αποθηκευμένο "εγκέφαλο" από προηγούμενες εκπαιδεύσεις.
 - `NeuralNetwork.mutate(cars[i].brain, 0.4);`: Εφαρμόζει τη μετάλλαξη στον εγκέφαλο κάθε αυτοκινήτου, εκτός από το πρώτο.
 - **animate():** Η κύρια συνάρτηση του παιχνιδιού, η οποία καλείται συνεχώς μέσω του `requestAnimationFrame`.
 - **Λογική Εξέλιξης:**
 1. `for (...) { ... update(...) }`: Ενημερώνει όλα τα αυτοκίνητα (A.I. και κυκλοφορίας).
 2. `bestCar = cars.find((c) => ...);`: Βρίσκει το καλύτερο αυτοκίνητο. **Κριτήριο:** Το αυτοκίνητο με τη χαμηλότερη τιμή y έχει ταξιδέψει πιο μακριά προς τα πάνω (καθώς ο άξονας y αυξάνεται προς τα κάτω στον καμβά).
 3. `carCtx.translate(0, -bestCar.y + carCanvas.height * 0.7);`: Μετακινεί την κάμερα για να ακολουθεί το καλύτερο αυτοκίνητο.

4. `carCtx.globalAlpha = 0.2`:: Κάνει όλα τα αυτοκίνητα ημιδιαφανή για να ξεχωρίζει το καλύτερο.
 5. `bestCar.draw(carCtx, true)`:: Σχεδιάζει το καλύτερο αυτοκίνητο χωρίς διαφάνεια και με ορατούς τους αισθητήρες του.
 6. `Visualizer.drawNetwork(...)`: Οπτικοποιεί τον εγκέφαλο του καλύτερου αυτοκινήτου.
- **Σκοπός:** Αυτή η διαδικασία ονομάζεται "**Προσαρμογή του Φυσικού Επιπέδου (Natural Selection)**". Το καλύτερο αυτοκίνητο "επιβιώνει" και ο "εγκέφαλός" του αναπαράγεται και μεταλλάσσεται για την επόμενη γενιά, οδηγώντας σε συνεχή βελτίωση.
 - `function save()` και `function discard()`: Αυτές οι συναρτήσεις είναι για την αποθήκευση και διαγραφή του καλύτερου "εγκεφάλου" στον `localStorage`, επιτρέποντας στον χρήστη να συνεχίσει την εκπαίδευση αργότερα.